



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Mock Exam - 3 (Wohoooo! Level Up!)

- *14 Questions*

QUESTION 1 OF 14



Weight: 6

You are an administrator preparing your environment to deploy a Kubernetes cluster using kubeadm. Adjust the following network parameters on the system to the following values, and make sure your changes persist reboots:

`net.ipv4.ip_forward = 1`

`net.bridge.bridge-nf-call-iptables = 1`

- ☐ `net.ipv4.ip_forward` is set to 1
- ☐ `net.bridge.bridge-nf-call-iptables` is set to 1

Solution

Use `sysctl` to adjust system parameters and make sure they persist across reboots. To set the required `sysctl` parameters and make them persistent:

```
echo 'net.ipv4.ip_forward = 1' >> /etc/sysctl.conf
echo 'net.bridge.bridge-nf-call-iptables = 1' >> /etc/sysctl.conf
sysctl -p
```

To verify:

```
sysctl net.ipv4.ip_forward
sysctl net.bridge.bridge-nf-call-iptables
```

QUESTION 2 OF 14

Weight: 8

Create a new service account with the name `pvviewer`. Grant this Service account access to `list` all PersistentVolumes in the cluster by creating an appropriate cluster role called `pvviewer-role` and ClusterRoleBinding called `pvviewer-role-binding`.

Next, create a pod called `pvviewer` with the image: `redis` and serviceAccount: `pvviewer` in the default namespace.

- ☐ ServiceAccount: pvviewer
- ☐ ClusterRole: pvviewer-role
- ☐ ClusterRoleBinding: pvviewer-role-binding
- ☐ Pod: pvviewer
- ☐ Is the pod configured to use ServiceAccount pvviewer?

Solution

Pods authenticate to the API Server using ServiceAccounts. If the serviceAccount name is not specified, the default service account for the namespace is used during a pod creation.

Reference: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>

Now, create a service account `pvviewer`:

```
kubectl create serviceaccount pvviewer
```

To create a clusterrole:

```
kubectl create clusterrole pvviewer-role --resource=persistentvolumes --verb=list
```

To create a clusterrolebinding:

```
kubectl create clusterrolebinding pvviewer-role-binding --clusterrole=pvviewer-role --serviceaccount=default:pvviewer
```

Solution manifest file to create a new pod called `pvviewer` as follows:

```
---
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: pvviewer
    name: pvviewer
spec:
  containers:
  - image: redis
    name: pvviewer
  # Add service account name
  serviceAccountName: pvviewer
```

QUESTION 3 OF 14

Weight: 6

Create a StorageClass named `rancher-sc` with the following specifications:

The provisioner should be `rancher.io/local-path`.

The volume binding mode should be `WaitForFirstConsumer`.

Volume expansion should be enabled.

- ☐ StorageClass rancher-sc is present
- ☐ Provisioner is rancher.io/local-path
- ☐ VolumeBindingMode is WaitForFirstConsumer

Solution

Use `kubectl create storageclass` and specify the required provisioner and parameters. Create a manifest file `rancher-sc.yaml`:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: rancher-sc
provisioner: rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Apply the file on the cluster:

```
kubectl apply -f rancher-sc.yaml
```

To verify:

```
kubectl get storageclass rancher-sc -o yaml
```

QUESTION 4 OF 14

Weight: 8

Create a ConfigMap named `app-config` in the namespace `cm-namespace` with the following key-value pairs:

```
ENV=production
LOG_LEVEL=info
```

Then, modify the existing Deployment named `cm-webapp` in the same namespace to use the `app-config` ConfigMap by setting the environment variables `ENV` and `LOG_LEVEL` in the container from the ConfigMap.

- ☐ ConfigMap `app-config` is created
- ☐ Deployment uses the `app-config` ConfigMap for variable `ENV` and `LOG_LEVEL`
- ☐ Are the environment variables reflected in the deployment?
- ☐ ConfigMap has proper `ENV` value
- ☐ ConfigMap has proper `LOG_LEVEL` value

Solution

Use `kubectl create configmap` to create the ConfigMap and `kubectl set env` to modify the deployment to use the ConfigMap.

1. Create the ConfigMap:

```
kubectl create configmap app-config -n cm-namespace \
--from-literal=ENV=production \
--from-literal=LOG_LEVEL=info
```

2. Patch the deployment to use the config:

```
kubectl set env deployment/cm-webapp -n cm-namespace \
--from=configmap/app-config
```

QUESTION 5 OF 14

Weight: 8

Create a PriorityClass named `low-priority` with a value of 50000. A pod named `lp-pod` exists in the namespace `low-priority`. Modify the pod to use the priority class you created. Recreate the pod if necessary.

- ☐ Is the PriorityClass low-priority created?
- ☐ Low priority class value is set properly to 50000
- ☐ Pod lp-pod uses the low-priority PriorityClass

Solution

Use `kubectl apply` to create the PriorityClass, then edit the existing pod to add the priority class.

1. Create the priority class manifest file `pc.yaml`

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 50000
globalDefault: false
description: "Low priority class"
```

2. Apply the manifest file

```
kubectl apply -f pc.yaml
```

3. Inspect the pod definition to be able to recreate it:

```
kubectl get pod lp-pod -n low-priority -o yaml
```

4. Create and update the new yaml `lp-pod.yaml` for the pod after adding the priority class:

```
# vi lp-pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: lp-pod
  namespace: low-priority
spec:
  priorityClassName: low-priority
  containers:
  - name: nginx
    image: nginx
```

4. Delete and recreate the pod using the configured priority class

```
kubectl delete pod lp-pod -n low-priority
kubectl apply -f lp-pod.yaml
```

QUESTION 6 OF 14

Weight: 8

A pod called `np-test-1` and a service called `np-test-service` have been deployed in the `default` namespace. A `default-deny` NetworkPolicy is currently blocking all ingress traffic to pods in this namespace, which is why the service is unreachable.

Create a new NetworkPolicy named `ingress-to-nptest` in the `default` namespace that allows ingress traffic from **all sources** to the `np-test-1` pod on port `80`.

Important: Don't delete any current objects deployed.

- ☐ Important: Don't Alter Existing Objects! (default-deny NetworkPolicy must still exist)
- ☐ NetworkPolicy: Is the port correct?
- ☐ NetworkPolicy: Is it applied to the correct Pod?
- ☐ Is ingress traffic to np-test-1 pod reachable on port 80?

Solution

A `default-deny` NetworkPolicy is already in place blocking all ingress traffic. You need to create a second NetworkPolicy that explicitly allows ingress to the `np-test-1` pod on port 80 from any source.

1. Inspect the existing `default-deny` policy to understand the current state:

```
kubectl get netpol -n default
kubectl describe netpol default-deny
```

2. Create a manifest file `ingress-to-nptest.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ingress-to-nptest
  namespace: default
spec:
  podSelector:
    matchLabels:
      run: np-test-1
  policyTypes:
  - Ingress
  ingress:
  - ports:
    - protocol: TCP
      port: 80
```

Leaving the `from:` field empty under `ingress` means traffic is allowed from **all sources**.

3. Apply the policy:

```
kubectl apply -f ingress-to-nptest.yaml
```

4. Verify traffic is now allowed:

```
kubectl run test-conn --image=busybox --restart=Never --rm -it -- wget -qO- -T 5 http://np-test-service
```

You should see the nginx welcome page in the output.

Solution manifest file to create a network policy `ingress-to-nptest` as follows:

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ingress-to-nptest
  namespace: default
spec:
  podSelector:
    matchLabels:
      run: np-test-1
  policyTypes:
  - Ingress
  ingress:
  - ports:
    - protocol: TCP
      port: 80
```

QUESTION 7 OF 14

Weight: 12

Taint the worker node `node01` to be Unschedulable. Once done, create a pod called `dev-redis`, image `redis:alpine`, to ensure workloads are not scheduled to this worker node. Finally, create a new pod called `prod-redis` and image: `redis:alpine` with toleration to be scheduled on `node01`.

key: `env_type`, value: `production`, operator: `Equal` and effect: `NoSchedule`

- ☐ key = `env_type`
- ☐ value = `production`
- ☐ effect = `NoSchedule`
- ☐ Is pod 'dev-redis' (no tolerations) not scheduled on node01?
- ☐ Is the 'prod-redis' to running on node01?

Solution

To add taints on the `node01` worker node:

```
kubectl taint node node01 env_type=production:NoSchedule
```

Now, deploy `dev-redis` pod and to ensure that workloads are not scheduled to this `node01` worker node.

```
kubectl run dev-redis --image=redis:alpine
```

To view the node name of recently deployed pod:

```
kubectl get pods -o wide
```

Solution manifest file to deploy new pod called `prod-redis` with toleration to be scheduled on `node01` worker node.

```
---
apiVersion: v1
kind: Pod
metadata:
  name: prod-redis
spec:
  containers:
    - name: prod-redis
      image: redis:alpine
  tolerations:
    - effect: NoSchedule
      key: env_type
      operator: Equal
      value: production
```

To view only `prod-redis` pod with less details:

```
kubectl get pods -o wide | grep prod-redis
```

QUESTION 8 OF 14

Weight: 6

A PersistentVolumeClaim named `app-pvc` exists in the namespace `storage-ns`, but it is not getting bound to the available PersistentVolume named `app-pv`.

Inspect both the PVC and PV and identify why the PVC is not being bound and fix the issue so that the PVC successfully binds to the PV. Do not modify the PV resource.

☐ Is PVC correctly bound to PV?

Solution

Use `kubectl describe` and check the PVC and PV details to identify the issue.

1. Inspect both resources:

```
kubectl describe pv app-pv
kubectl describe pvc app-pvc -n storage-ns
```

2. Note that there is accessModes mismatch between PV and PVC objects. Flush the PVC in a yaml file to be able to update it:

```
kubectl get pvc app-pvc -n storage-ns -o yaml > pvc.yaml
```

3. Update the access mode on `pvc.yaml`:

```
# vi pvc.yaml

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
  namespace: storage-ns
spec:
  accessModes:
    - ReadWriteOnce      # fix access mode to match PV object
  resources:
    ...
```

4. Delete and re-create the pvc object

```
kubectl delete pvc -n storage-ns app-pvc
kubectl apply -f pvc.yaml
```

4. Verify the status of the PVC is showing as `Bound`:

```
kubectl get pvc app-pvc -n storage-ns
```

QUESTION 9 OF 14

Weight: 8

A kubeconfig file called `super.kubeconfig` has been created under `/root/CKA`. There is something wrong with the configuration. Troubleshoot and fix it.

- ☐ Fix `/root/CKA/super.kubeconfig`

Solution

Verify host and port for `kube-apiserver` are correct.

Open the `super.kubeconfig` in vi editor.

Change the 9999 port to 6443 and run the below command to verify:

```
kubectl cluster-info --kubeconfig=/root/CKA/super.kubeconfig
```

QUESTION 10 OF 14

Weight: 10

We have created a new deployment called `nginx-deploy`. Scale the deployment to 3 replicas. Has the number of replicas increased? Troubleshoot and fix the issue.

☐ Does the deployment have 3 replicas?

Solution

Use the command `kubectl scale` to increase the replica count to 3.

```
kubectl scale deploy nginx-deploy --replicas=3
```

The `controller-manager` is responsible for scaling up pods of a replicaset. If you inspect the control plane components in the `kube-system` namespace, you will see that the `controller-manager` is not running.

```
kubectl get pods -n kube-system
```

The command (binary name) running inside the `controller-manager` pod has a typo — the letter `l` has been replaced with the numeral `1` in `kube-controller-manager`.

Fix the binary name in the manifest file:

```
sed -i 's/kube-contro1ler-manager/kube-controller-manager/g' /etc/kubernetes/manifests/kube-controller-manager.yaml
```

The kubelet detects the change automatically and restarts the pod within ~20 seconds. Wait for the `kube-controller-manager` pod to show as Running:

```
kubectl get pods -n kube-system | grep controller-manager
```

Once the controller-manager is running, the deployment scale will take effect. Verify with:

```
kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-deploy	3/3	3	3	6m2s

QUESTION 11 OF 14

Weight: 6

Create a Horizontal Pod Autoscaler (HPA) `api-hpa` for the deployment named `api-deployment` located in the `api` namespace. The HPA should scale the deployment based on a custom metric named `requests_per_second`, targeting an average value of 1000 requests per second across all pods. Set the minimum number of replicas to 1 and the maximum to 20.

Note: Deployment named `api-deployment` is available in `api` namespace. Ignore errors due to the metric `requests_per_second` not being tracked in `metrics-server`.

- ☐ Is `api-hpa` HPA deployed in `api` namespace?
- ☐ Is `api-hpa` configured for metric `requests_per_second`?

Solution

Under `/root/` folder you will find a yaml file `api-hpa.yaml`. Update the yaml file as per task given.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
  namespace: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-deployment
  minReplicas: 1
  maxReplicas: 20
  metrics:
  - type: Pods
    pods:
      metric:
        name: requests_per_second
      target:
        type: AverageValue
        averageValue: "1000"
```

Use below command

```
kubect1 create -f api-hpa.yaml
```

QUESTION 12 OF 14

Weight: 6

Configure the `web-route` to split traffic between `web-service` and `web-service-v2`. The configuration should ensure that 80% of the traffic is routed to `web-service` and 20% is routed to `web-service-v2`.

Note: `web-gateway`, `web-service`, and `web-service-v2` have already been created and are available on the cluster.

- ☐ Is the web-route deployed as HTTPRoute?
- ☐ Is the route configured to gateway `web-gateway`?
- ☐ Is the route configured to service `web-service`?

Solution

Copy the below YAML file to the terminal and create a HTTP Route.

```
kubectl create -n default -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-route
  namespace: default
spec:
  parentRefs:
    - name: web-gateway
      namespace: default
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: web-service
          port: 80
          weight: 80
        - name: web-service-v2
          port: 80
          weight: 20
EOF
```

EOF

QUESTION 13 OF 14

Weight: 4

One application, `webpage-server-01`, is currently deployed on the Kubernetes cluster using Helm. A new version of the application is available in a Helm chart located at `/root/new-version`.

Validate this new Helm chart, then install it as a new release named `webpage-server-02`. After confirming the new release is installed, uninstall the old release `webpage-server-01`.

- ☐ Is the new version app deployed?
- ☐ Is the old version app uninstalled?

Solution

In this task, we will use the `helm` commands. Here are the steps:

1. Use the `helm ls` command to list the Helm releases in the `default` namespace.

```
helm ls -n default
```

2. Validate the Helm chart using the `helm lint` command:

```
cd /root/  
helm lint ./new-version
```

3. Install the new version of the application as a new release named `webpage-server-02`:

```
helm install webpage-server-02 ./new-version
```

4. Uninstall the old release `webpage-server-01` using the following command:

```
helm uninstall webpage-server-01 -n default
```

QUESTION 14 OF 14

Weight: 4

While preparing to install a CNI plugin on your Kubernetes cluster, you typically need to confirm the cluster-wide Pod network CIDR. Identify the Pod subnet configured for the cluster (the value specified under `podSubnet` in the `kubeadm` configuration). Output this CIDR in the format `x.x.x.x/x` to a file located at `/root/pod-cidr.txt`.

Note: Use the cluster-wide `podSubnet` from the `kubeadm-config` ConfigMap, not the per-node CIDR from `kubectl get node`.

☐ Is the cluster-wide Pod CIDR network correctly written to the file?

Solution

Look for the `podSubnet` value in the `kubeadm-config` ConfigMap in the `kube-system` namespace: `kubectl -n kube-system get configmap kubeadm-config -o yaml | grep podSubnet`. Important: This cluster-wide CIDR (e.g., /16) is different from the per-node CIDR shown by `kubectl get node -o jsonpath='{.spec.podCIDR}'` (e.g., /24).

To identify the cluster-wide Pod subnet, inspect the `kubeadm-config` ConfigMap, which contains the `ClusterConfiguration` used during `kubeadm init`:

```
kubectl -n kube-system get configmap kubeadm-config -o yaml | grep podSubnet
# podSubnet: 172.17.0.0/16
```

Save just the CIDR value to `/root/pod-cidr.txt`:

```
kubectl -n kube-system get configmap kubeadm-config -o yaml \
| awk '/podSubnet:/{print $2}' > /root/pod-cidr.txt
```

Verify:

```
cat /root/pod-cidr.txt
# 172.17.0.0/16
```

Note:

Be careful not to confuse **Cluster PodCIDR** vs **Node PodCIDR**:

- Cluster = Entire pool
Cluster PodCIDR (big range: 172.17.0.0/16)
Find it with: `kubectl get cm kubeadm-config -n kube-system -o yaml`
- Node = Single slice
Node PodCIDR (small slice: 172.17.0.0/24)
Find it with: `kubectl get node <name> -o jsonpath='{.spec.podCIDR}'`